“Linux Concurrent Producer–Consumer System Using Semaphores”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

M. Ramya Sree	**2520030094**
S. Lohitha	**2520030109**
D. Teja Sadwika	**2520030536**

Under the Esteemed Guidance of

Dr. K. Anusha
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"Linux Concurrent Producer–Consumer System Using Semaphores"** is a record of Bonafide work of **M Ramya Sree - 2520030094 , S Lohitha - 2520030109, D Teja Sadwika - 2520030536** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

M Ramya Sree – 2520030094
S Lohitha – 2520030109
D Teja Sadwika - 2520030536

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **"Linux Concurrent Producer–Consumer System Using Semaphores"** is a bonafide work done and submitted by **M Ramya Sree - 2520030094, S Lohitha - 2520030109, and D Teja Sadwika - 2520030536,**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Dr. K. Anusha**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

M Ramya Sree 2520030094

S Lohitha 2520030109

D Teja Sadwika 2520030536

Abstract

The Producer–Consumer problem is a fundamental problem in operating systems that demonstrates the challenges of coordinating multiple processes or threads that share a common resource. When a producer generates data and a consumer processes it simultaneously, improper coordination can result in race conditions, buffer overflow and buffer underflow.

This project focuses on the design and implementation of a Linux-based concurrent Producer–Consumer system using POSIX threads, semaphores and mutex synchronization. The main application uses a bounded circular buffer to temporarily store items produced by the producer thread. The consumer thread retrieves these items while synchronization mechanisms ensure that shared resources are accessed safely.

The system uses counting semaphores to maintain the availability of empty and filled buffer positions. A mutex provides mutual exclusion while accessing shared buffer data. POSIX threads allow the producer and consumer to execute concurrently.

In addition to the main simulation, the application integrates process management, inter-process communication, system information retrieval, file management and memory analysis. These modules demonstrate important Linux operating system functionalities through a menu-driven C application.

The project is developed using Ubuntu Linux, GCC compiler and POSIX/Linux system interfaces. The expected outcome is a working application that demonstrates concurrency, synchronization, resource management and communication in a Linux environment.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	

		5.4 Module Description	
		5.5 Input and Output Design	
6	**Implementation**		**30–45**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**62–67**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**79–83**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**84–86**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

An operating system manages computer resources and provides an environment in which multiple programs can execute. Modern operating systems support concurrent execution through processes and threads.

When multiple threads access shared data, coordination becomes important. Without proper synchronization, one thread may modify data while another thread is accessing it, leading to unexpected results.

The Producer–Consumer problem is a classical synchronization problem in which one thread produces items and another thread consumes them using a shared buffer.

This project implements the Producer–Consumer problem in Ubuntu Linux using C programming and POSIX synchronization mechanisms. It also integrates other operating system functionalities into one menu-driven application.

1.2 Problem Statement

When producer and consumer threads operate simultaneously using a shared buffer, improper synchronization can cause several problems.

- Buffer overflow when the producer adds items to a full buffer.
- Buffer underflow when the consumer tries to remove items from an empty buffer.
- Race conditions when multiple threads access shared data without proper protection.
- Inconsistent data due to simultaneous modifications.
- Improper resource management during concurrent execution.

Therefore, a synchronization-based solution is required to coordinate producer and consumer operations and ensure correct execution.

1.3 Scope of the Project

The project provides a Linux-based application that demonstrates important operating system functionalities.

The main scope includes:

1.Producer–Consumer simulation using a bounded circular buffer.

2.Thread creation and concurrent execution.

3.Synchronization using POSIX semaphores and mutexes.

4.Process creation and execution.

5.Inter-process communication using pipes and shared memory.

6.Retrieval of Linux system information.

7.File handling and activity logging.

8.Memory management simulation using page tables, TLB and FIFO replacement.

The application is developed in C and executed in an Ubuntu Linux environment.

1.4 Objectives of the Project

- To implement concurrent execution using POSIX threads.
- To demonstrate synchronization using semaphores and mutexes.
- To prevent buffer overflow and underflow.
- To implement a bounded circular buffer.
- To demonstrate process creation and termination.
- To implement communication between processes.
- To perform file operations and retrieve system information.
- To simulate memory address translation and page replacement.
- To integrate all modules into a single menu-driven application.

1.5 Importance of the Application

Synchronization is important in operating systems because multiple threads may share resources simultaneously.

This project provides a practical demonstration of how threads coordinate their activities, how shared resources are protected and how processes communicate.

It helps students understand concurrent execution, synchronization, process management and resource management through practical implementation.

1.6 Target Users / Audience

- Computer Science and Engineering students.
- Operating Systems learners.
- Students studying Linux and POSIX programming.
- Beginners interested in concurrent programming.
- Students learning process and memory management.

CHAPTER 2 – OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Process Management

A process is a program in execution. Linux provides mechanisms to create new processes, execute programs and wait for process completion.

The project demonstrates process creation and execution using Linux process-management functions.

Thread Management

A thread is the smallest unit of execution within a process. Multiple threads can execute concurrently and share the same process memory.

The project creates separate producer and consumer threads using POSIX threads.

Process Synchronization

Synchronization coordinates the execution of multiple threads to prevent conflicts when accessing shared resources.

The project uses semaphores and mutexes to coordinate access to the shared buffer.

Inter-Process Communication

Inter-Process Communication (IPC) allows processes to exchange information.

The project demonstrates communication using pipes and shared memory.

File Management

Linux provides file operations to create, read, write and retrieve information about files.

The project maintains an activity log and demonstrates file-related operations.

Memory Management

Memory management involves allocating, tracking and accessing memory resources.

The project includes a memory-management simulator that demonstrates logical address translation, page tables, TLB operations and FIFO page replacement.

2.2 Existing Methods / Algorithms

The Producer–Consumer problem can be addressed using different synchronization techniques. Common approaches include:

- Busy waiting: Threads repeatedly check whether a resource is available.
- Mutex locks: Ensure only one thread accesses a critical section at a time.
- Semaphores: Control access to shared resources using counters.
- Bounded-buffer approach: Restricts the number of items that can be stored in a buffer.

The proposed implementation uses a bounded circular buffer with counting semaphores and a mutex.

2.3 Proposed Approach

The proposed system uses two concurrent threads:

Producer: Generates items and inserts them into the shared circular buffer.

Consumer: Retrieves items from the shared buffer and processes them.

Two counting semaphores are maintained:

- Empty semaphore: Tracks available buffer positions.
- Full semaphore: Tracks occupied buffer positions.

A mutex protects the critical section during buffer insertion and removal.

The application also integrates supporting Linux functionalities through separate modules.

2.4 Advantages of the Proposed Approach

- Prevents buffer overflow and underflow.
- Supports concurrent execution.
- Provides safe access to shared data.
- Demonstrates practical synchronization.
- Uses a circular buffer for efficient storage.
- Integrates several Linux functionalities.
- Provides a simple menu-driven interface.
- Helps understand operating system concepts practically.

CHAPTER 3 – SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Requirement
Processor	Intel or AMD processor
RAM	Minimum 4 GB recommended
Storage	Minimum 1 GB free space
Keyboard	Standard keyboard
Display	Standard monitor

3.2 Software Requirements

Software	Purpose
Ubuntu Linux	Operating environment
C Language	Application development
GCC	Compilation
GNU Make	Build automation
POSIX Libraries	Threads and synchronization
Linux Terminal	Execution and testing

3.3 Operating System Environment – Ubuntu

The project is developed and executed using Ubuntu Linux.

Ubuntu provides the required Linux system interfaces, POSIX thread support, semaphores, process-management functions and file operations.

3.4 Development Tools and Compilers

GCC Compiler: Used to compile C source files into an executable program.

GNU Make: Used to compile and manage multiple source files through a Makefile.

Nano Editor: Used to create and edit source files through the terminal.

Ubuntu Terminal: Used to execute Linux commands, compile the project and test the application.

CHAPTER 4 – TECHNOLOGY AND TOOLS USED

4.1 C Programming Language

C is used to develop the application because it provides direct access to system-level programming features and supports Linux/POSIX interfaces.

4.2 GCC Compiler

GCC is used to compile the project source files.

Example:

gcc -Wall -Wextra -std=c11 -pthread

The -pthread option enables POSIX thread support.

4.3 Linux System Calls and APIs

The following system interfaces are used in the project.

Function / API	Purpose
fork()	Creates a child process
exec family	Executes another program
waitpid()	Waits for child process completion
pipe()	Creates a communication channel
mmap()	Maps memory for shared-memory communication
open()	Opens a file
read()	Reads file contents
write()	Writes file contents
lseek()	Changes file position
fstat()	Retrieves file information
uname()	Retrieves system information
getpid()	Returns process ID
pthread_create()	Creates a thread
pthread_join()	Waits for a thread
sem_init()	Initializes a semaphore
sem_wait()	Decreases semaphore count or waits
sem_post()	Increases semaphore count

pthread_mutex_lock() Locks a mutex

pthread_mutex_unlock() Unlocks a mutex

4.4 POSIX Threads and Synchronization

POSIX threads allow multiple threads to execute within a process.

The producer and consumer are implemented as separate threads.

Counting semaphores coordinate empty and full buffer positions, while a mutex protects the shared buffer during critical operations.

4.5 Linux Commands and Utilities

The following commands are used during development and execution.

Command	Purpose
pwd	Displays current directory
ls	Lists files
cd	Changes directory
mkdir	Creates directory
nano	Edits source files
gcc	Compiles programs
make	Builds the application
make clean	Removes previous build output
./linux_app	Executes the application

4.6 Other Tools and Libraries Used

- Standard C library.
- POSIX thread library.
- POSIX semaphore library.
- Linux system interfaces.
- GNU Make build utility.

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The application follows a modular architecture.

The user interacts with the main menu, which connects to different functional modules.

The modules use Linux/POSIX interfaces to perform their operations.

The major components are:

1. Main menu module.
2. Producer–Consumer module.
3. Process management module.
4. Communication module.
5. System information module.
6. File management module.
7. Memory analysis module.

5.2 System Workflow

The application follows the workflow below:

1. The user launches the application.
2. The main menu is displayed.
3. The user selects a required operation.
4. The corresponding module is executed.
5. The module performs its operations using Linux/POSIX interfaces.
6. The results are displayed in the terminal.
7. The user can return to the main menu.
8. The application exits when the user selects the exit option.

5.3 Flowchart / Data Flow

Overall application flow:

START

↓

Initialize application

↓

Display main menu

↓

Read user choice

↓

Select corresponding module

↓

Execute selected operation

↓

Display result

↓

Return to main menu

↓

Check exit option

↓

END

Producer–Consumer workflow:

START

↓

Initialize bounded buffer

↓

Initialize empty and full semaphores

↓

Initialize mutex

↓

Create producer and consumer threads

↓

Producer waits for an empty slot

↓

Producer locks mutex and inserts an item

↓

Producer unlocks mutex and signals full semaphore

↓

Consumer waits for a filled slot

↓

Consumer locks mutex and removes an item

↓

Consumer unlocks mutex and signals empty semaphore

↓

Repeat until all items are processed

↓

Join threads and release resources

↓

END

5.4 Module Description

Module 1: Producer–Consumer Simulation

This is the main module of the application.

It creates producer and consumer threads that share a bounded circular buffer.

Semaphores control buffer availability, and a mutex protects shared buffer access.

Module 2: Process Management

This module demonstrates process creation and execution.

It uses fork() to create a child process, an exec function to execute a Linux command and waitpid() to wait for child completion.

Module 3: Communication Tools

This module demonstrates IPC.

Pipe communication transfers data between processes through a communication channel.

Shared memory allows processes to access a common memory region.

Module 4: System Information

This module displays information about the Linux environment.

It retrieves process information, operating system details, kernel version and machine architecture.

Module 5: File Management

This module demonstrates file handling operations.

It allows users to add activities to a log, view stored activities, retrieve file information and check file positions.

Module 6: Memory Analysis

This module simulates memory-management operations.

It demonstrates logical address translation, page-table lookup, TLB hits and misses, page faults and FIFO page replacement.

Note: This module is a simulator and does not modify the actual Linux virtual-memory subsystem.

5.5 Input and Output Design

The application accepts input through the Linux terminal.

Examples of inputs include:

- Main menu selection.
- Buffer capacity.
- Number of items to produce.
- Communication options.
- File operations.
- Logical memory addresses.

The output includes:

- Producer and consumer operations.
- Process IDs and execution status.
- Communication messages.
- System information.
- File information and activity logs.
- Memory-management results.

CHAPTER 6 – IMPLEMENTATION

6.1 Module-wise Implementation

The project is divided into multiple C source files.

File	Functionality
main.c	Main menu and module selection
common.h	Shared declarations
producer_consumer.c	Producer–Consumer simulation
process_manager.c	Process creation and management
ipc_manager.c	Pipe and shared-memory communication
system_info.c	Linux system information
file_manager.c	File handling and activity logging
memory_manager.c	Memory-management simulation
Makefile	Compilation and build management

6.2 Algorithm / Pseudocode

Producer Algorithm

1. Start producer thread.
2. Repeat for the required number of items.
3. Wait for an empty buffer position.
4. Lock the mutex.
5. Insert the item into the circular buffer.
6. Update the buffer index.
7. Unlock the mutex.
8. Signal the full semaphore.
9. Repeat until all items are produced.
10. Terminate producer thread.

Consumer Algorithm

1. Start consumer thread.
2. Repeat for the required number of items.
3. Wait for a filled buffer position.
4. Lock the mutex.
5. Remove an item from the circular buffer.

6. Update the buffer index.
7. Unlock the mutex.
8. Signal the empty semaphore.
9. Repeat until all items are consumed.
10. Terminate consumer thread.

6.3 Core C Implementation

The main simulation uses a shared data structure containing:

- Buffer array.
- Buffer capacity.
- Total number of items.
- Producer index.
- Consumer index.
- Produced item count.
- Consumed item count.
- Empty semaphore.
- Full semaphore.
- Mutex.

The producer and consumer functions operate on this shared structure.

The application allocates memory dynamically for the buffer and releases it after the simulation.

6.4 Linux System Calls / APIs Used

The implementation uses Linux and POSIX interfaces to perform process management, communication, synchronization and file operations.

The functions are organized into separate source files to improve modularity and maintainability.

6.5 Process / Thread Implementation

The application demonstrates both process-based and thread-based execution.

Process management uses process creation and execution functions.

Thread management uses pthread_create() to create producer and consumer threads and pthread_join() to wait for their completion.

6.6 IPC / Synchronization Implementation

Pipe communication is used to transfer messages between processes.

Shared memory provides a common memory region for process communication.

For Producer–Consumer synchronization:

- The empty semaphore tracks available positions.
- The full semaphore tracks occupied positions.
- The mutex provides mutual exclusion.

These mechanisms prevent simultaneous unsafe access to the shared buffer.

6.7 Memory Management Implementation

The memory module demonstrates logical address translation.

It includes:

- Page number calculation.
- Offset calculation.
- Page-table lookup.
- TLB hit and miss tracking.
- Page fault tracking.
- FIFO page replacement.

The module provides a simplified educational simulation of memory-management concepts.

6.8 User Interface Implementation

The application uses a terminal-based menu interface.

Users can select modules through numbered options.

The main menu connects all six functionalities and provides a simple way to execute and test each module.

CHAPTER 7 – EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Open the terminal and navigate to the project directory.

cd ~/Producer_Consumer_Linux

Clean previous compilation files:

make clean

Compile the application:

make

If compilation completes successfully, the executable file linux_app is generated.

7.2 Execution Commands

Run the application using:

./linux_app

The main menu displays all six modules.

7.3 Sample Inputs and Outputs

Test 1: Producer–Consumer Simulation

Input:

- Buffer capacity: 3
- Number of items: 5

Expected result:

- Five items are produced.
- Five items are consumed.
- The circular buffer is used.
- The execution summary displays equal produced and consumed counts.

Example summary:

========== EXECUTION SUMMARY ==========

Buffer Capacity : 3

Items Requested : 5

Items Produced : 5

Items Consumed : 5

Simulation completed successfully!

Shared resources released successfully.

Test 2: Process Management

The process management module creates a child process and executes a Linux command.

Expected result:

- Parent process ID is displayed.
- Child process ID is displayed.
- Child executes the selected command.
- Parent waits for child completion.

Test 3: Communication Tools

The communication module demonstrates pipe and shared-memory communication.

Expected result:

- Parent sends a message through a pipe.
- Child receives the message.
- Shared data can be updated and accessed by the communicating processes.

Test 4: System Information

The system information module displays operating system and process-related details.

Expected result:

- Process ID.
- Parent process ID.
- User ID.
- Group ID.
- Operating system.
- Kernel version.
- Machine architecture.

Test 5: File Management

The file management module supports adding and viewing activity logs.

Expected result:

- Activity is written to the log.
- Stored activities can be viewed.
- File information is displayed.
- File position can be checked.

Test 6: Memory Analysis

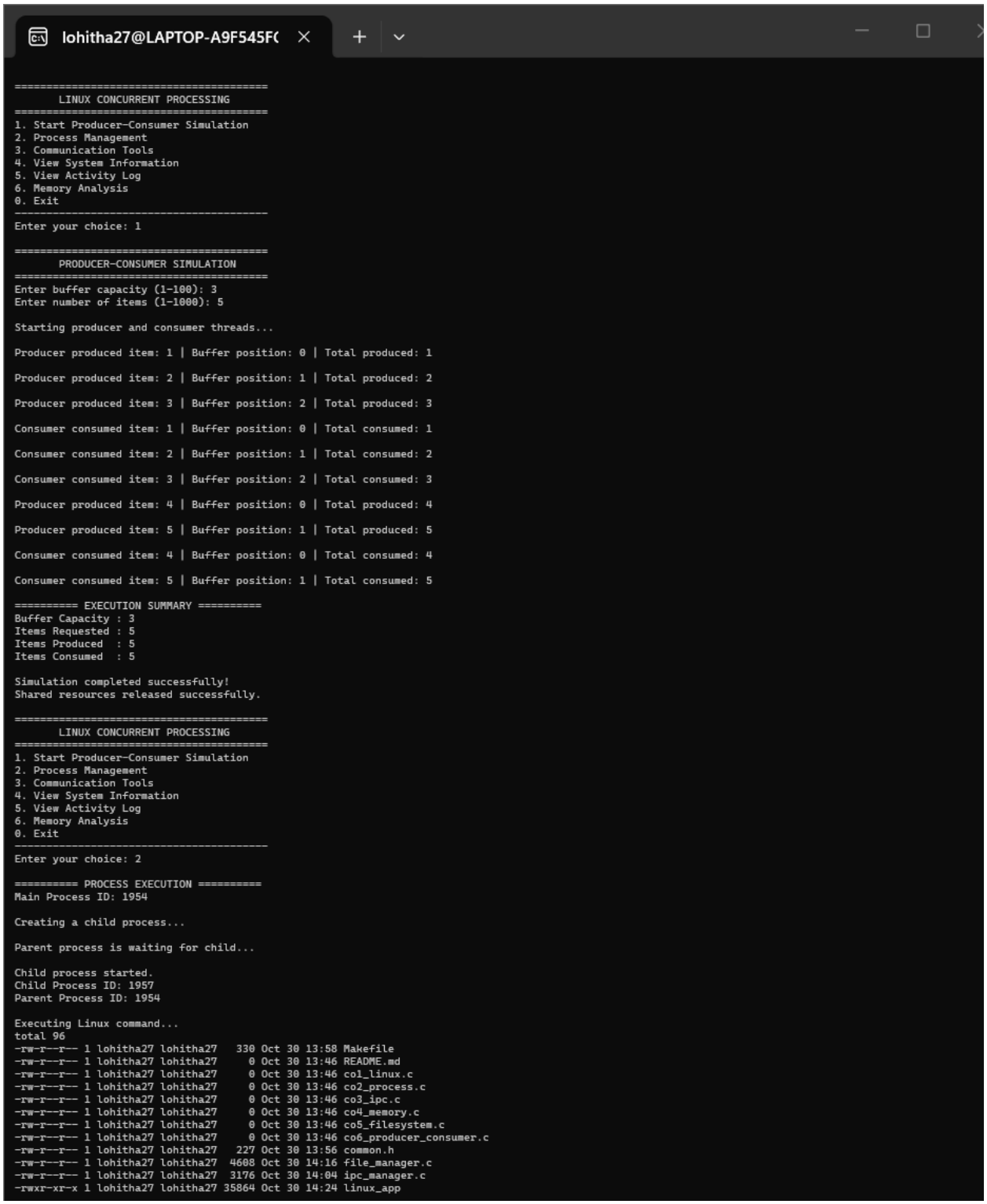
The memory module demonstrates logical address translation and memory statistics.

Expected result:

- Page number and offset are calculated.

- Page table is checked.
- TLB hits and misses are tracked.
- Page faults are recorded.
- FIFO replacement is demonstrated.

7.4 Screenshots of Execution



7.5 Demonstration of OS Concepts

The project demonstrates important operating system functionalities through practical implementation.

- Concurrent execution using POSIX threads.
- Synchronization using semaphores and mutexes.
- Process creation using fork().
- Program execution using the exec family.
- Process waiting using waitpid().
- IPC using pipes and shared memory.
- File operations using Linux file interfaces.
- System information retrieval.
- Simulated memory-management operations.

CHAPTER 8 – CONCLUSION

8.1 Summary of the Project

The Linux Concurrent Producer–Consumer System Using Semaphores was developed as a menu-driven C application in Ubuntu Linux.

The main functionality demonstrates concurrent execution using producer and consumer threads that share a bounded circular buffer.

POSIX semaphores and mutex synchronization ensure that shared buffer operations are performed safely.

The application also integrates process management, inter-process communication, system information retrieval, file handling and memory-management simulation.

8.2 Objectives Achieved

The project successfully implements the following functionalities:

1. Producer–Consumer simulation using POSIX threads.
2. Synchronization using counting semaphores.
3. Mutual exclusion using mutexes.
4. Bounded circular buffer implementation.
5. Process creation and execution.
6. Pipe and shared-memory communication.

7. Linux system information retrieval.
8. File management and activity logging.
9. Memory-management simulation.

8.3 Results Achieved

The application was compiled and executed in Ubuntu Linux.

The Producer–Consumer simulation was tested with different buffer capacities and item counts.

The produced and consumed counts matched during successful test executions.

The supporting modules were integrated into a single application and tested through the main menu.

The project demonstrates practical implementation of Linux/POSIX programming concepts.

8.4 Skills and Learning Outcomes

Through this project, we gained practical knowledge of:

- C programming in Linux.
- Concurrent programming.
- POSIX thread management.
- Semaphore and mutex synchronization.
- Process management.
- Inter-process communication.
- Linux file operations.
- Memory-management concepts.
- Compilation and debugging using GCC.
- Modular software development.

FUTURE ENHANCEMENTS

The project can be enhanced through:

- Multiple producer and consumer threads.
- Configurable execution delays.
- Performance measurement and comparison.
- Additional synchronization techniques.
- Improved error handling.
- Automated testing.

- More detailed execution reports.

CHAPTER 9 – REFERENCES

9.1 Books

1. Abraham Silberschatz, Peter B. Galvin and Greg Gagne, *Operating System Concepts*.
2. William Stallings, *Operating Systems: Internals and Design Principles*.
3. Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*.

9.2 Research and Learning Resources

1. Operating Systems course materials.
2. POSIX thread programming concepts.
3. Producer–Consumer synchronization problem references.

9.3 Linux / GNU Documentation

1. Linux Manual Pages – fork(2), exec(3), waitpid(2).
2. Linux Manual Pages – pipe(2), mmap(2).
3. Linux Manual Pages – open(2), read(2), write(2).
4. Linux Manual Pages – pthread_create(3), pthread_join(3).
5. Linux Manual Pages – sem_init(3), sem_wait(3), sem_post(3).
6. GCC Documentation.
7. GNU Make Manual.

9.4 Online Resources

1. https://man7.org/linux/man-pages/
2. https://gcc.gnu.org/onlinedocs/

3. https://www.gnu.org/software/make/manual/
4. https://pubs.opengroup.org/onlinepubs/9699919799/